

Context-Free Grammar (CFG)



**SCS 2312 Computational Models and Programming Language
Concepts**

Session Objectives

- Understand why grammar is needed
- Learn CFG components
- Perform derivations
- Draw parse trees
- Identify ambiguity

How does a compiler know whether a program is correct or incorrect?

- Does it understand meaning?
- Or does it check rules?

Example

✓ `int x = 10;`

✗ `= int 10 x;`

**Can a programming language
have only a limited number of
programs?**

Can a programming language have only a limited number of programs?

No.

A programming language has infinitely many valid programs

Why is it infinite?

- Unlimited identifiers
- Unlimited numeric values
- Unlimited combinations of statements

How can we describe infinitely many programs using a finite description?"

We describe the language using rules

- What is allowed in a programming language.
- What is not allowed in a programming language
- How symbols can be combined within the language

“These rules together are called **Grammar.**”

What is Grammar?

A finite set of rules

What do we mean by rules?

Rules are instructions that say:

- What symbols are allowed
- what order they can appear
- How larger structures are built.

What is Grammar?

➤ Describes valid strings of a language

A string can be,

- Number
- Expression
- Statement
- Full program

What is Grammar?

Using a **finite number of rules**, grammar
can generate **infinite programs**

Example: Derivation of a Number

<digit> → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<number> → <digit> | <digit><number>

Example: Generating a Number

<number>

$\Rightarrow$ <digit><number>

$\Rightarrow$ 3<number>

$\Rightarrow$ 3<digit>

$\Rightarrow$ 34

Generate 246

<number>

⇒ <digit><number>

⇒ 2<number>

⇒ 2<digit><number>

⇒ 24<number>

⇒ 24<digit>

⇒ 246

CONTEXT-FREE GRAMMAR (CFG)

CFG is the formal way we describe the syntax of programming languages

$$G = (V, \Sigma, P, S)$$

V — Non-terminal Symbols

“Non-terminals are not actual symbols of the language.”

Examples:

- **<expr>** means an expression
- **<stmt>** means a statement
- **<term>** means part of an expression

Terminal Symbols (Σ)

“Terminal symbols are the actual symbols of the language.”

In programming languages, terminals are things like:

- keywords
- operators
- identifiers
- numbers

Terminal Symbols (Σ)

Examples:

- +
- =
- *
- A, B, C

Production Rules (P)

“These rules tell us how non-terminals can be replaced.”

Each rule has the form:

Non-terminal $\rightarrow$ symbols

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$

Start Symbol (S)

“This is called the start symbol, and we denote it by S.”

“This is where derivation always begins.”

What is BNF?

- BNF stands for **Backus–Naur Form**
- BNF is simply a standard way of writing grammar rules

$$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$$
$$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{id} \rangle$$
$$\langle \text{id} \rangle \rightarrow \text{A} \mid \text{B} \mid \text{C}$$

What is BNF?

Example

$A = B + C$

$\langle \text{assign} \rangle$

$\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{id} \rangle + \langle \text{id} \rangle$

$\Rightarrow A = B + C$

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

Example

A = B + C * A

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

Example

$\langle \text{assign} \rangle$

$\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle + \langle \text{term} \rangle$

$\Rightarrow A = \langle \text{term} \rangle + \langle \text{term} \rangle$

$\Rightarrow A = \langle \text{factor} \rangle + \langle \text{term} \rangle$

$\Rightarrow A = B + \langle \text{term} \rangle$

$\Rightarrow A = B + \langle \text{term} \rangle * \langle \text{factor} \rangle$

$\Rightarrow A = B + \langle \text{factor} \rangle * \langle \text{id} \rangle$

$\Rightarrow A = B + \langle \text{id} \rangle * A$

$\Rightarrow A = B + C * A$

$A = B + C * A$

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

PARSE TREES

- A hierarchical representation of a derivation
- Shows how grammar rules are applied.

Root → **Start symbol**

Internal nodes → **Non-terminals**

Leaves → **Terminals**

Draw the parse tree for $A = B + C$

$\langle \text{assign} \rangle$

$\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{id} \rangle + \langle \text{id} \rangle$

$\Rightarrow A = B + C$

Draw the parse tree for $A = (B + C) + A$

$\langle \text{assign} \rangle$

$\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle + \langle \text{term} \rangle$

$\Rightarrow A = \langle \text{factor} \rangle + \langle \text{term} \rangle$

$\Rightarrow A = (\langle \text{expr} \rangle) + \langle \text{term} \rangle$

$\Rightarrow A = (\langle \text{expr} \rangle + \langle \text{term} \rangle) + \langle \text{term} \rangle$

$\Rightarrow A = (\langle \text{term} \rangle + \langle \text{term} \rangle) + \langle \text{term} \rangle$

$\Rightarrow A = (\langle \text{factor} \rangle + \langle \text{term} \rangle) + \langle \text{term} \rangle$

$\Rightarrow A = (\langle \text{id} \rangle + \langle \text{term} \rangle) + \langle \text{term} \rangle$

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

$\Rightarrow A = (B + \langle \text{term} \rangle) + \langle \text{term} \rangle$

$\Rightarrow A = (B + \langle \text{factor} \rangle) + \langle \text{term} \rangle$

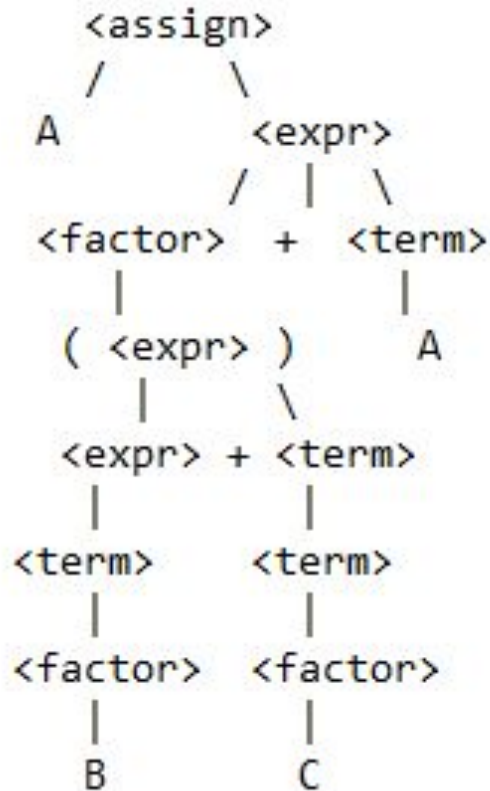
$\Rightarrow A = (B + \langle \text{id} \rangle) + \langle \text{term} \rangle$

$\Rightarrow A = (B + C) + \langle \text{term} \rangle$

$\Rightarrow A = (B + C) + \langle \text{id} \rangle$

$\Rightarrow A = (B + C) + A$

Draw the parse tree for $A = (B + C) + A$



<assign> $\rightarrow$ **<id>** = **<expr>**

<expr> $\rightarrow$ **<expr>** + **<term>** | **<term>**

<term> $\rightarrow$ **<term>** * **<factor>** | **<factor>**

<factor> $\rightarrow$ (**<expr>**) | **<id>**

<id> $\rightarrow$ **A** | **B** | **C**

Draw the parse tree for $P = Q + R * S + T$

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow \langle \text{id} \rangle$

$\langle \text{id} \rangle \rightarrow Q \mid R \mid S \mid T$

Draw the parse tree for $P = Q + R * S + T$

```
<assign>
⇒ <id> = <expr>
⇒ P = <expr>
⇒ P = <expr> + <term>
⇒ P = <expr> + <term> + <term>
⇒ P = <term> + <term> + <term>
⇒ P = <factor> + <term> + <term>
⇒ P = <id> + <term> + <term>
⇒ P = Q + <term> + <term>
⇒ P = Q + <term> * <factor> + <term>
⇒ P = Q + <factor> * <factor> + <term>
⇒ P = Q + <id> * <factor> + <term>
⇒ P = Q + R * <factor> + <term>|
⇒ P = Q + R * <id> + <term>
⇒ P = Q + R * S + <term>
⇒ P = Q + R * S + <factor>
⇒ P = Q + R * S + <id>
⇒ P = Q + R * S + T
```

Chomsky Hierarchy of Grammars

- Grammars have different power levels, just like machines.
- This classification is called the Chomsky Hierarchy

Type 3 → Regular → Finite Automaton

Type 2 → Context-Free → Pushdown Automaton

Type 1 → Context-Sensitive → LBA

Type 0 → Unrestricted → Turing Machine

TYPE 3 — REGULAR GRAMMAR

- Regular grammar are used to describe simple patterns

Examples

- keywords
- identifiers
- numbers

TYPE 2 — CONTEXT-FREE GRAMMAR

- Can handle nested structures & balanced expressions

Examples

- arithmetic expressions
- if-else blocks
- function definitions

TYPE 2 — CONTEXT-FREE GRAMMAR

“Programming languages mainly use Type 2 — Context-Free Grammar.”

why,

- Nested blocks
- Parentheses
- Expressions inside expressions

TYPE 1 & TYPE 0

Type 1 — Context-Sensitive Grammar

- More powerful than Context-Free Grammar
- Can handle rules where context matters

Type 0 — Unrestricted Grammar

- Most powerful grammar type
- Equivalent to a Turing Machine

AMBIGUITY & OPERATOR PRECEDENCE

- Ambiguity occurs when the same string can have more than one parse tree

More than one parse tree,

→ More than one structure

→ More than one meaning

- Dangerous for compilers → may produce incorrect results

Example

‘I saw the man with the telescope.’

Who has the telescope?

“The sentence is **ambiguous** because it has two meanings.”

How Do We SOLVE Ambiguity?

‘We solve ambiguity by controlling the structure’

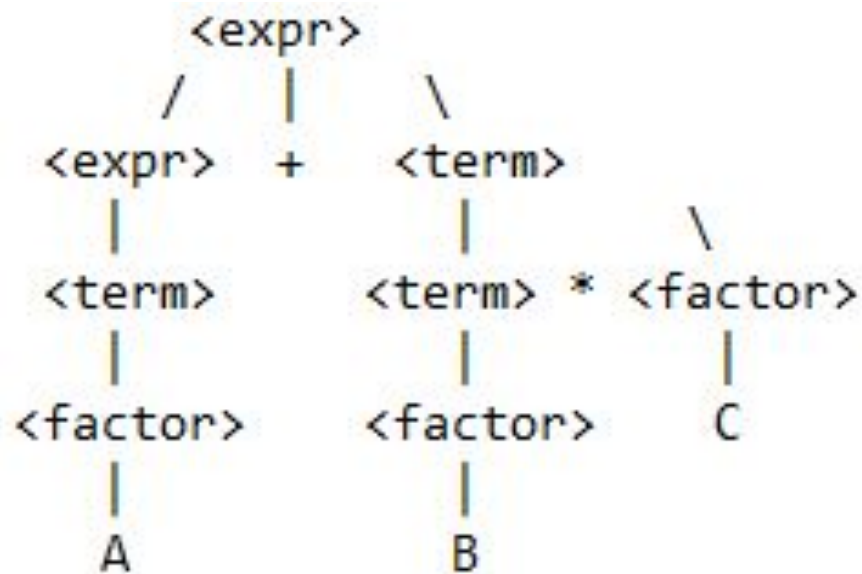
$$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$$
$$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$$
$$\langle \text{factor} \rangle \rightarrow \text{id}$$

Solution: Grammar to Resolve Ambiguity

We can remove ambiguity by:

- separating operators
- using different non-terminals
- controlling tree structure

Parse Tree for $A + B * C$



Introduction to Associativity

Operator Associativity

- Decides order of evaluation when operators have the same precedence

$A - B - C = (A - B) - C \rightarrow$ Left associative

$A = B = C = A = (B = C) \rightarrow$ Right associative

Precedence + Associativity

Grammar Decides Both

- **Precedence** → which operator is done first ($*$ vs $+$)
- **Associativity** → order when operators are equal ($A - B - C$)
- Both are enforced by grammar structure

Associativity of Operators

Operator	Description	Associativity	Example
$\wedge$	Exponential	Right to left	a^b
$*$ /	Multiplication and division	Left to right	$a*b$
$+$ -	Addition and subtraction	Left to right	$A-b$